

Algorithms for Closure & Symmetry Analysis

This chapter collects algorithmic methods for analyzing closure properties and symmetry structure in finite digital objects, with an emphasis on practical decision procedures and reproducible tooling. The central computational tasks are clone membership, generator discovery, canonicalization, equivalence checking, and complexity assessment. In each case, the goal is not only to decide a property, but also to quantify the reliability of the decision procedure through an error profile $\varepsilon(r)$, where the parameter r denotes the chosen resolution, resource budget, or relaxation level. In practice, $\varepsilon(r)$ summarizes soundness and precision, together with observed SAT/SMT success rates under the selected encoding.

A first family of methods addresses clone membership. Given a Boolean function, circuit, or finite operation, one asks whether it belongs to a specified clone or closure class. Two standard computational encodings are especially useful: SAT-based encodings, which reduce membership to satisfiability of a constraint system, and BDD-based encodings, which exploit canonical decision-diagram structure when the function admits compact representation. In the SAT setting, the membership question is translated into a search for witnesses that violate the defining closure conditions. In the BDD setting, membership can often be checked by structural containment or by testing whether the function can be generated from the prescribed basis under composition. These methods also support generator discovery: rather than merely deciding membership, one searches for a small set of generators whose closure reproduces the target object or class.

Canonicalization is the next major theme. For graph-structured objects and Boolean functions, canonical forms provide a way to compare instances modulo symmetry. Graph isomorphism algorithms supply a canonical labeling or a certificate of equivalence for graph representations of circuits and related combinatorial structures. For Boolean functions, NPN canonicalization is the standard symmetry-reduction problem: one seeks a representative under negation of inputs, permutation of inputs, and negation of the output. Heuristic canonicalization procedures are often necessary in practice, especially when exact search becomes expensive. Typical heuristics include partition refinement, local search over symmetry candidates, signature-based pruning, and incremental refinement from coarse invariants to finer ones. These methods do not replace exact algorithms, but they can dramatically reduce the search space before a final decision step.

Equivalence checking is the unifying decision problem behind these constructions. Two objects are equivalent if they compute the same function, induce the same closure, or lie in the same symmetry orbit under the relevant action. In a SAT-based workflow, equivalence checking is often implemented by constructing a miter or difference formula and asking whether a counterexample exists. In a BDD-based workflow, equivalence can be reduced to pointer equality after canonical reduction, provided the representation is sufficiently compact. For graph-based objects, equivalence may be checked through isomorphism testing or through canonical labels derived from an isomorphism engine. The practical choice among these methods depends on the size of the instance, the expected symmetry, and the cost of the underlying representation.

The complexity-theoretic perspective clarifies why multiple algorithmic strategies are needed. Exact clone membership and equivalence problems can be computationally difficult, and their tractability often depends on parameters such as arity, treewidth, circuit depth, or the size of the symmetry group. Parameterized complexity provides a useful language for this analysis: a problem may be intractable in general while remaining fixed-parameter tractable with respect to a structural measure. This viewpoint is particularly relevant for symmetry analysis, where the presence of large automorphism groups can either help or hinder computation. On the one hand, symmetry can be exploited to compress search; on the other hand, it can create many equivalent branches that must be pruned carefully.

A practical implementation therefore benefits from a layered pipeline. One begins with coarse invariants and cheap filters, then applies SAT, BDD, or graph-isomorphism machinery only when the preliminary tests do not settle the instance. This ordering is especially useful when the benchmark family contains many easy cases mixed with a smaller number of hard instances. In such a pipeline, the reported $\varepsilon(r)$ should be interpreted together with the chosen encoding, the pruning strategy, and the stopping criterion, since these design choices affect both the observed success rate and the reliability of the final answer.

The chapter therefore treats algorithms and complexity as two sides of the same design problem. A good implementation should expose the tradeoff between exactness, scalability, and certification. The metric $\varepsilon(r)$ is used here as a compact summary of that tradeoff: lower values indicate better soundness and precision at the chosen resource level, while higher SAT/SMT success rates indicate that the encoding is effective on the target benchmark family. In a reproducible workflow, these quantities are reported together with instance size, symmetry profile, and solver configuration.

The accompanying artifact is a CLI toolkit with benchmarks. The toolkit is intended to support batch evaluation of clone membership, canonicalization, and equivalence-checking pipelines on a curated set of examples. A typical command-line workflow includes loading an instance, selecting an encoding strategy, running the solver or canonicalizer, and exporting a machine-readable report containing the decision result, runtime, resource usage, and the observed $\varepsilon(r)$ summary. Benchmark output should be stable enough to support regression testing and cross-method comparison, so that improvements in heuristics or encodings can be measured against a fixed baseline.

Taken together, these methods provide a practical bridge between the abstract closure and symmetry theory developed earlier in the book and the executable analyses needed in later case studies. They show how clone-theoretic questions, symmetry reduction, and equivalence checking can be turned into concrete algorithms, while preserving a disciplined account of correctness and computational cost.